Plateforme e-commerce SPLINEDGE - Cahier des charges

Ce document sert pour les ateliers. Chaque section liste les points à discuter et à valider. La rédaction détaillée continue après validation de ces points.

> Ce cahier des charges est la référence unique de la cible produit. Le `MVP`, la `V1` et la `V2+` sont des livraisons successives de cette même cible. Ce ne sont pas des périmètres séparés.

Table des matières

1. Contexte du projet
   - Vue d'ensemble
   - Problème à résoudre
   - Objectifs produit
   - Hypothèses, contraintes et limites
   - Parties prenantes et rôles
   - Parcours minimum `MVP`
   - Périmètre par phase
2. Fonctionnalités et parcours
   - Vue fonctionnelle détaillée
   - Pages prévues
   - Principes UX/UI
3. Règles métier
   - Devise
   - FIFO
   - Fournisseurs et achats
   - Prix de vente multi-lots
   - Multi-villes
   - Alertes
   - Commandes et statuts
4. Cadre technique
   - Données principales et intégrations
   - Stack technique
   - Stockage des images
   - Architecture logique
   - Conteneurisation
   - API, sécurité et conformité
   - Tests et validation

5. Organisation du projet
   — Ordre de développement
   — Planning indicatif
   — Livrables
   — Gouvernance
   — Risques
   — Plan de démarrage
6. Glossaire

---

1. Contexte du projet

1.1 Vue d'ensemble

SPLINEDGE développe une plateforme e-commerce. La plateforme doit gérer une chaîne complète : approvisionnement, stock et vente.

Le contexte métier est complexe :

— les achats et les coûts sont en `EUR` ;

— les ventes sont en `MAD` ;

— les fournisseurs peuvent vendre à `crédit` ou demander un `paiement avant réception` ;

— les stocks sont suivis par `lots` ;

— la sortie des lots suit la règle `FIFO` ;

— la vente peut se faire unité par unité ;

— le prix de vente peut changer d'un lot à l'autre ;

— le prix de référence peut être recalculé avec une `moyenne pondérée` ;

— le stock peut être réparti sur plusieurs sites ;

— les dates de péremption et les alertes doivent être prises en compte ;

— le code-barres aide les équipes terrain à limiter les erreurs.

La première étape est un `MVP`. Son but est de valider les flux critiques métier et technique avant d'élargir le périmètre.

1.2 Problème à résoudre

Le produit doit gérer correctement les prix, les stocks et les achats fournisseurs quand :

— le taux de change évolue ;

— plusieurs lots du même article ont des coûts différents ;

— le stock est réparti sur plusieurs villes ;

— les dates de péremption et la disponibilité doivent être prises en compte ;

— une même commande, ou une même ligne, doit puiser dans plusieurs lots tout en respectant le `FIFO` et en gardant une traçabilité cohérente sur le coût, les quantités et les mouvements ;

— les fournisseurs imposent des modalités différentes : `crédit` ou `paiement anticipé` avant réception ;

— les prix de vente peuvent varier entre lots ; il faut donc définir comment calculer le prix affiché ou le prix de référence ;

— les ventes se font unité par unité ; chaque sortie doit donc respecter le `FIFO` au niveau du lot consommé.

1.3 Objectifs produit

Le produit doit :

— proposer une solution e-commerce solide et évolutive ;

— maîtriser le parcours `catalogue -> commande -> stock -> livraison` ;

— assurer la cohérence entre coût, prix et stock, par lot et par ville ;

— poser une base technique prête pour une future mise en production.

1.4 Hypothèses, contraintes et limites

— Les versions `V1` et `V2+` suivront les retours du `MVP`.

— Le `MVP` contient l'essentiel pour valider le métier et la technique.

— Les engagements de type `SLA`, l'exploitation `24h/24` et le support de production sont hors `MVP`.

> Le `MVP` sert d'abord à apprendre, valider et sécuriser les flux critiques. Il ne porte pas encore d'engagement d'exploitation industrielle.

1.5 Parties prenantes du projet

Les acteurs du projet sont :

— le sponsor ou décideur métier ;

— le responsable produit, qui porte les besoins et la priorisation ;

— l'équipe de développement ;

— le superviseur, qui valide et arbitre.

1.6 Rôles métier dans l'application

Rôle (code)	Explication
`admin`	Gère le catalogue, les commandes, les stocks et les alertes au niveau global.
`customer`	Parcourt le site, remplit le panier, passe commande et suit sa commande.
`seller`	Opérateur local. Il gère la réception, le scan, l'inventaire et les mouvements de stock sur le terrain.

1.7 Parcours minimum à couvrir en `MVP`

Le `MVP` doit couvrir au minimum :

1. la connexion selon le rôle ;
2. la consultation du catalogue et la mise au panier ;
3. le passage de commande ;
4. la mise à jour du stock avec gestion des lots et du `FIFO` ;

5. le suivi de commande ;
6. l'utilisation du scan en réception ou en vente selon le cas.

1.8 Périmètre par phase

Le détail exact par fonctionnalité est dans la section 2. Les tags `[MVP]`, `[V1]` et `[V2+]` y font foi. Cette règle évite la duplication.

Résumé d'orientation, non exhaustif :

— `MVP` : boucle courte pour valider les flux critiques métier et technique, sans engagement `SLA` de production ;

— `V1` : après retours sur le `MVP`, accent sur l'exploitation, la fiabilité, la sécurité et l'observabilité avec logs et métriques, en plus des éléments déjà tagués `[V1]` ;

— `V2+` : évolutions différenciantes et automatisation plus poussée, arbitrées après le `MVP`.

---

2. Fonctionnalités et parcours

2.1 Vue fonctionnelle détaillée

Chaque bloc garde le même sens que le document source. Les champs sont présentés de façon plus claire : priorité, acteurs, description, user story et critères d'acceptation.

Connexion sécurisée, sessions `JWT`, renouvellement, `RBAC` et journal des actions sensibles `[MVP]`

— **Acteurs :** tous les rôles

— **Priorité :** `MVP`

— **Description :** le système gère une authentification sécurisée, le renouvellement des sessions, le contrôle d'accès par rôle (`RBAC`) et la trace des actions sensibles.

— **User story :** en tant qu'utilisateur autorisé, je veux accéder seulement aux écrans de mon rôle avec une session sécurisée.

— **Critères d'acceptation :**

1. Un utilisateur avec le bon rôle accède aux écrans prévus. Sinon, l'API retourne `403`.
2. Sans jeton valide, ou avec un jeton expiré, l'API retourne `401` sans fuite de données métier.
3. Un renouvellement de jeton valide fournit une nouvelle paire de jetons. Un jeton invalide est refusé proprement.
4. Les actions sensibles, comme la connexion et les échecs, sont tracées dans le journal.

Catalogue, fiche produit, panier persistant, commande et confirmation `[MVP]`

— **Acteurs :** `customer`

— **Priorité :** `MVP`

— **Description :** le client consulte le catalogue, ouvre une fiche produit, garde son panier, passe commande et reçoit une confirmation.

— **User story :** en tant que client, je veux parcourir les produits et commander sans perdre mon panier.

— **Critères d'acceptation :**

1. Le catalogue gère les catégories, la recherche et la pagination.
2. La fiche produit affiche le prix, le stock et la disponibilité en cohérence avec les règles métier.
3. Le panier reste conservé entre les visites.
4. Le passage de commande aboutit à un message de confirmation. Le parcours reste utilisable de bout en bout.

Administration des produits, catégories, commandes, stocks et alertes `[MVP]`

— **Acteurs :** `admin`

— **Priorité :** `MVP`

— **Description :** l'administration centralise la gestion du catalogue, des commandes, des stocks, des lots et des alertes.

— **User story :** en tant qu'`admin`, je veux piloter le back-office depuis un seul espace de travail.

— **Critères d'acceptation :**

1. Le `CRUD` des produits et des catégories fonctionne.
2. Le suivi des commandes et la mise à jour des statuts suivent le workflow cible.
3. La vue du stock par ville et par lot est exploitable pour les opérations courantes.
4. Les alertes sont consultables et déclenchent des actions correctives côté back-office.

Enregistrement des lots d'achat en `EUR` avec conversion en `MAD` et taux figé à l'entrée `[MVP]`

— **Acteurs :** `admin`

— **Priorité :** `MVP`

— **Description :** l'`admin` enregistre un lot acheté en `EUR`. Le système convertit le coût en `MAD` et fige le taux à l'entrée.

— **User story :** en tant qu'`admin`, je veux garder un coût traçable en `MAD` pour chaque lot.

— **Critères d'acceptation :**

1. Après la création d'un lot, le taux `EUR -> MAD` est figé et relisible.
2. Le coût en `MAD` du lot est calculé et traçable.
3. Chaque import est identifiable : qui, quand et quel lot.
4. L'historique des taux utilisés reste consultable.

Gestion des fournisseurs et des achats (`crédit` vs `paiement avant réception`) `[V1]`

— **Acteurs :** `admin`

— **Priorité :** `V1`

— **Description :** le système gère les fournisseurs, les commandes d'achat et les différences entre achat à crédit et paiement avant réception.

— **User story :** en tant qu'`admin`, je veux suivre les achats fournisseurs avec des statuts, des montants et une traçabilité claire.

— **Critères d'acceptation :**

1. Chaque lot, ou mouvement d'achat, peut être rattaché à un fournisseur et à une commande fournisseur, ou équivalent, avec des statuts cohérents comme `commandé`, `payé partiellement`, `payé intégralement` et `reçu`.
2. En cas d'achat à crédit, le système enregistre la dette ou les échéances selon le modèle retenu. Le stock n'est disponible à la vente qu'après la règle métier choisie, comme réception ou validation, à trancher.
3. En cas de paiement avant réception, le paiement fournisseur est tracé. La marchandise n'entre en stock qu'à la réception, par scan ou par saisie validée.
4. Les montants en `EUR` et `MAD`, ainsi que les dates, restent auditables : qui, quand et quel mouvement.

Prix de vente par lot et prix de référence par moyenne pondérée quand plusieurs lots coexistent `[V1]`

— **Acteurs :** `admin`, système

— **Priorité :** `V1`

— **Description :** chaque lot peut avoir son propre prix de vente. Le système calcule aussi un prix de référence quand plusieurs lots du même produit coexistent.

— **User story :** en tant qu'`admin`, je veux gérer des prix différents par lot tout en gardant un prix catalogue cohérent.

— **Critères d'acceptation :**

1. Chaque lot peut porter un prix de vente unitaire distinct en `MAD` quand le métier le demande.
2. Quand un même produit existe sur plusieurs lots avec des prix différents, le prix affiché, ou prix catalogue de référence, suit la moyenne pondérée décrite en section 3. La pondération utilise les quantités restantes par lot, sauf arbitrage contraire.
3. Chaque recalcul après mouvement de stock, comme vente ou réception, est reproductible et vérifiable avec un jeu de données de test.
4. Les arrondis et l'affichage côté client sont uniformes et alignés avec les règles de devise.

Gestion des stocks par lots et consommation `FIFO` à la vente `[MVP]`

— **Acteurs :** `admin`, `seller`, système

— **Priorité :** `MVP`

— **Description :** le stock est suivi par lot. La consommation suit le `FIFO`, y compris quand une ligne de commande prend plusieurs lots.

— **User story :** en tant qu'opérateur ou système, je veux consommer les lots dans le bon ordre et garder une traçabilité complète.

— **Critères d'acceptation :**

1. Si une ligne de commande utilise plusieurs lots, l'ordre de consommation va du plus ancien au plus récent.
2. En vente unité par unité, chaque unité vendue consomme d'abord le lot le plus ancien disponible sur le périmètre retenu, entrepôt ou ville, conformément au `FIFO`.
3. Chaque sortie crée un mouvement de stock lié à la commande.
4. En cas d'annulation ou de correction, il ne doit pas y avoir de stock négatif silencieux.

5. Les quantités sont suivies au niveau du lot.

Multi-villes, stock par entrepôt et alertes (`rupture`, stock faible, péremption) `[MVP]`

— **Acteurs :** `admin`

— **Priorité :** `MVP`

— **Description :** le stock est localisé par ville ou par entrepôt. Le back-office gère aussi les alertes de rupture, de stock faible et de péremption.

— **User story :** en tant qu' `admin`, je veux voir le stock par lieu et agir vite sur les alertes.

— **Critères d'acceptation :**

1. Le stock est localisé par ville ou entrepôt et visible dans l'administration.
2. Les seuils d'alerte sont réglables par l'équipe.
3. Quand le stock passe sous le seuil, une alerte est créée ou mise à jour selon les règles définies.
4. Les alertes critiques sont visibles dans le back-office.

Scan code-barres en réception et en vente, avec lien produit, lot et opération `[MVP]`

— **Acteurs :** `seller`

— **Priorité :** `MVP`

— **Description :** le scan permet de retrouver la bonne entité et de lancer l'action prévue pendant une réception ou une vente.

— **User story :** en tant que `seller`, je veux scanner un code-barres pour travailler plus vite et éviter les erreurs.

— **Critères d'acceptation :**

1. Un code reconnu correspond à une seule entité attendue dans le contexte d'utilisation.
2. Un code inconnu affiche un message clair sans modifier le stock par erreur.
3. Le scan déclenche l'action prévue, par exemple réception ou vente.
4. Le système prend en charge une douchette ou un lecteur. L'option caméra web ou mobile dépend du périmètre retenu.

Paiement de commande (`MVP` : simulé ou Stripe selon décision projet ; `V1` : renforcement) `[MVP / V1]`

— **Acteurs :** `customer`

— **Priorité :** `MVP / V1`

— **Description :** le client paie sa commande. Le `MVP` peut utiliser un paiement simulé ou Stripe selon la décision projet. La `V1` renforce le dispositif.

— **User story :** en tant que client, je veux payer ma commande sans double validation ni incohérence.

— **Critères d'acceptation :**

1. À la création, la commande a un statut initial cohérent avec le workflow `MVP`.
2. Le total en `MAD` est égal à la somme des lignes.
3. Après paiement, simulé ou confirmé, il n'y a pas de double validation incohérente.

4. Le traitement est idempotent pour éviter les doubles paiements. En `V1`, les webhooks du prestataire et l'idempotence sont documentés et testés.

Paiement réel stabilisé et observabilité avec logs et métriques `[V1]`

— **Acteurs :** `admin`, équipe technique

— **Priorité :** `V1`

— **Description :** la `V1` stabilise l'intégration de paiement réel et améliore l'observabilité.

— **User story :** en tant qu'équipe projet, nous voulons diagnostiquer vite les incidents de paiement.

— **Critères d'acceptation :**

1. L'intégration de paiement, Stripe ou équivalent, est jugée stable en recette et en préproduction.
2. La journalisation et les métriques permettent un diagnostic rapide des incidents courants.
3. Les événements de paiement restent traçables et cohérents avec les statuts de commande.

Recommandations avancées, `BI` temps réel étendue et automatisation supply `[V2+]`

— **Acteurs :** sponsor, responsable produit

— **Priorité :** `V2+`

— **Description :** la phase `V2+` ouvre des fonctions plus avancées et plus différenciantes.

— **User story :** en tant que sponsor ou responsable produit, je veux arbitrer des briques avancées après les retours du `MVP`.

— **Critères d'acceptation :**

1. Le périmètre et les priorités sont arbitrés après les retours sur le `MVP`. Il n'y a pas d'engagement fonctionnel sur ces briques dans le `MVP`.
2. Chaque brique livrée possède ses propres critères d'acceptation validés pendant le cadrage `V2+`.

2.2 Pages prévues

Chaque page garde un nom court. Sa description indique son objectif principal.

Page	Objectif principal
`Connexion`	Authentification sécurisée par rôle. Il n'y a pas de mode visiteur pour protéger les fonctions métier. `MVP`.
`Accueil / catalogue`	Liste des produits, catégories, recherche et pagination.
`Fiche produit`	Afficher le prix, le stock et la disponibilité.
`Panier`	Garder un contenu persistant entre les visites.
`Paiement / confirmation`	Permettre le passage de commande et afficher un message de confirmation. `MVP` : simulé ou Stripe.
`Suivi de commande`	Afficher les statuts visibles pour le client.
`Tableau de bord admin`	Donner une synthèse opérationnelle.
`Produits / catégories (admin)`	Créer, modifier et supprimer les produits et catégories avec `CRUD`.
`Stocks / lots (admin)`	Montrer le stock par ville et par lot, le `FIFO`, l'import `EUR -> MAD` et, en `V1`, le prix de vente par lot.
`Fournisseurs / achats (admin)`	Gérer les fiches fournisseurs, les commandes d'achat, le suivi `crédit` vs `paiement anticipé` et le lien avec les lots. `V1`.
`Commandes (admin)`	Lister les commandes, voir le détail et changer les statuts.
`Alertes (admin)`	Lister et traiter les alertes de rupture, de stock faible et de péremption.
`Scan - réception / vente`	Afficher les écrans d'opération, le résultat et l'action lancée.

2.3 Principes UX/UI

L'interface doit respecter trois principes simples :

- le parcours reste simple et lisible ;
- les messages d'erreur sont clairs ;
- les écrans s'adaptent à chaque rôle.

---

3. Règles métier

> Règle d'or : aucun stock fantôme. Chaque mouvement important doit être explicite, traçable et cohérent.

3.1 Règles de devise : conversion, affichage et arrondis

- La conversion s'applique à l'entrée du lot.
- Chaque lot garde son propre taux. Un changement de taux plus tard ne modifie pas les lots déjà enregistrés.
- Les achats sont en `EUR`.
- Les ventes sont en `MAD`.

— Le coût en `MAD` du lot est calculé et traçable.

3.2 Règles `FIFO` : entrée et sortie de stock

— La vente consomme les lots du plus ancien au plus récent.

— Le `FIFO` est strict sur la date d'entrée du lot, ou sur un champ équivalent documenté.

— En vente unité par unité, chaque unité vendue retire la quantité sur le lot `FIFO` courant jusqu'à épuisement du lot.

— Ensuite, le système passe au lot suivant dans l'ordre chronologique.

— Le système ne saute pas un lot tant que le lot courant a du stock disponible, sauf règle d'indisponibilité comme péremption ou blocage.

— Chaque sortie doit rester explicable dans l'historique avec des mouvements liés.

3.3 Règles fournisseurs et conditions d'achat : `crédit` et `paiement avant réception`

— Chaque achat est rattaché à un fournisseur.

— Chaque achat suit au minimum un mode de règlement : `crédit` ou `paiement anticipé`.

— En `crédit`, le système trace la dette fournisseur et les échéances si elles existent.

— En `crédit`, la disponibilité du stock à la vente suit la règle `réception validée`, ou un autre jalon défini avec le métier et figé en recette.

— En `paiement avant réception`, le paiement peut être enregistré avant l'entrée physique en stock.

— En `paiement avant réception`, les quantités ne deviennent disponibles qu'après la réception, donc après le mouvement d'entrée.

— Les transitions comme `commandé -> payé -> reçu`, ou leurs variantes, utilisent des statuts à valeurs fixes.

— Il ne doit exister aucun stock sans mouvement explicite.

3.4 Règles de prix de vente multi-lots : moyenne pondérée

— Un même produit peut avoir plusieurs lots en parallèle.

— Ces lots peuvent avoir des prix de vente unitaires en `MAD` différents.

— Quand plusieurs lots d'un même produit sont en stock, le prix de référence affiché au catalogue, ou utilisé comme prix public par défaut, suit une moyenne pondérée.

— Cette moyenne pondérée se base sur les quantités restantes de chaque lot concerné.

```
P_ref = Σ (prix_vente_lot_i × qty_restante_i) / Σ (qty_restante_i)
```

— Si un lot n'a pas de prix de vente spécifique, le système doit utiliser soit le prix de vente par défaut du produit, soit le prix du dernier lot. Ce choix reste à trancher une seule fois puis à documenter ici après validation client.

— À chaque mouvement, vente, entrée ou ajustement, le prix de référence est recalculé ou mis à jour avec la même règle.

— Les arrondis, comme le nombre de décimales et la règle d'arrondi, sont fixes et identiques pour l'affichage et pour la facturation sommaire.

3.5 Règles multi-villes : disponibilité et transfert

— Le stock est localisé par ville ou par entrepôt.

— L'affichage `disponible` côté client suit la règle métier choisie.

— Si plusieurs interprétations sont possibles, ce point doit être tranché en atelier.

3.6 Règles d'alerte : seuils, fréquence et destinataires

— Les seuils sont réglables par l'équipe.

— Les cas critiques ressortent clairement dans l'administration.

3.7 Règles de commande et cycle de statuts

Le parcours cible des commandes est :

`draft` `->` `paid` `->` `preparing` `->` `shipped` `->` `delivered`

ou

`draft` `->` `paid` `->` `preparing` `->` `shipped` `->` `cancelled`

---

4. Cadre technique

4.1 Entités de référence

Le catalogue métier s'appuie sur les entités suivantes :

— **User** : identité, email, mot de passe hashé, rôle et dates ;

— **Category** : nom de catégorie ;

— **Product** : nom, `SKU`, prix d'achat en `EUR`, taux `MAD`, lien de catégorie, prix de vente par défaut en `MAD`, etc. ;

— **Supplier** : identité, coordonnées, conditions de paiement par défaut, `crédit` ou `anticipé`, à affiner, et statut ;

— **PurchaseOrder** ou **SupplierOrder** : fournisseur, dates, statuts comme `commandé`, `payé`, `reçu`, montants et lien vers les lots ou les réceptions générées ;

— **SupplierPayment** : optionnel selon le modèle. Il gère les acomptes, paiements, échéances et rapprochements avec la commande fournisseur ;

— **Warehouse** : entrepôt et ville ;

— **Lot** : produit, entrepôt, quantités, taux figé, dates d'entrée et de péremption, prix de vente unitaire en `MAD` sur ce lot et lien fournisseur ou commande d'achat quand c'est applicable ;

— **StockMovement** : type, quantité et lien vers une commande ou une autre référence ;

— **Cart**, **Order**, **OrderItem** : panier et commande ;

— **Alert** : type, gravité, message et état lu ou non ;

— **ExchangeRate** : devises, taux et date d'effet ;

— **PaymentEvent** : prestataire, type d'événement et idempotence.

4.2 Produit, lot, stock et commande

— **Product** porte les informations visibles par le client.

— Certains champs de **Product** servent de valeurs par défaut à la création d'un lot.

— Le prix affiché peut venir des lots via la moyenne pondérée décrite en section 3.

— **Lot** est la vérité opérationnelle pour les quantités, le coût au taux figé, le prix de vente sur ce lot, le rattachement fournisseur ou achat, le `FIFO` et la péremption.

— Les ventes et les mouvements s'appuient sur les lots, pas seulement sur la fiche produit.

— Le bloc fournisseur et achat modélise le décalage entre paiement et réception, ainsi que le crédit.

— Les règles exactes de disponibilité du stock restent celles de la section 3 une fois arbitrées avec le métier.

4.3 Relations et règles de cohérence

— Les identifiants sont uniques.

— Les dates de création et de mise à jour sont conservées.

— Les statuts utilisent des valeurs fixes documentées.

— Les événements importants laissent des traces.

4.4 Interfaces et intégrations externes

— Paiement : Stripe ou équivalent, selon la décision `MVP`.

— E-mails automatiques : confirmation et autres messages utiles.

— API de taux de change : optionnelle et hors `MVP` si elle n'est pas indispensable.

4.5 Stack technologique retenue

Couche	Technologie
Front	`Next.js`
Back	`NestJS`
Base de données	`PostgreSQL` + `Prisma`
Stockage fichiers	`MinIO`
Conteneurisation	`Docker` et `Docker Compose`
API	`REST` versionnée

4.6 Stratégie de stockage des images

> Les images ne seront pas stockées en `BLOB` dans `PostgreSQL`.

Nous utiliserons un stockage objet `MinIO` pour les fichiers. `PostgreSQL` gardera seulement les métadonnées.

Choix retenu :

— **Stockage des fichiers :** `MinIO`

— **Stockage en base avec PostgreSQL/Prisma :**

  — `id`

  - `objectKey`
  - `mimeType`
  - `size`
  - `ownerId`
  - `createdAt`
  - `updatedAt`

  Pourquoi ce choix :

- meilleure scalabilité pour les médias, même si les images deviennent volumineuses et nombreuses ;
- performances plus adaptées à la lecture et à l'écriture de fichiers ;
- base de données plus légère pour le backup, la restauration et les migrations ;
- intégration plus simple avec un `CDN`, du cache, des `URL` signées et des politiques d'expiration.

  Flux fonctionnel résumé :

1. L'utilisateur envoie l'image, ou demande une `URL` d'upload signée.
2. Le backend `NestJS` valide le fichier : taille, type `MIME` et sécurité.
3. Le fichier est stocké dans `MinIO`.
4. Le backend enregistre les métadonnées dans `PostgreSQL` via `Prisma`.
5. L'application consomme l'`URL` ou la clé pour afficher l'image côté `Next.js`.

4.7 Architecture logique

- Le site public et l'administration sont séparés côté usage.
- L'API est centrale.
- Les règles métier ne doivent pas rester dans de simples contrôleurs `HTTP`.
- La base est structurée avec des migrations versionnées.

4.8 Conteneurisation et environnements

Le projet s'exécute en conteneurs pour standardiser les environnements de développement, de test et de déploiement.

- **Technologies :** `Docker` et `Docker Compose`
- **Services minimaux :**
  - `frontend` pour `Next.js`
  - `backend` pour `NestJS`
  - `db` pour `PostgreSQL`
  - `minio` pour le stockage objet
- **Objectifs :**
  - réduire les écarts entre les machines locales et les environnements cibles ;
  - simplifier l'onboarding et l'exécution des tests ;
  - faciliter la reproductibilité des scénarios du `MVP`, puis de la `V1+`.

— **Configuration :**
  — variables d'environnement centralisées dans `.env` ;
  — volumes persistants pour `PostgreSQL` et `MinIO` ;
  — réseau interne dédié entre services.

4.9 Organisation des dossiers

```
frontend/
backend/
docs/
```

4.10 Exigences non fonctionnelles

— **Performance cible :** le temps de réponse doit rester adapté à l'usage réel.

— **Performance `MVP` :** le comportement doit être correct sur un environnement de démo. L'objectif indicatif est `p95 ≈ 500 ms` sur les parcours courants, catalogue et API, selon la machine.

— **Scalabilité :** la structure doit pouvoir grandir dès le `MVP`. La cible doit permettre d'ajouter des ressources ou des instances.

— **Fiabilité :** il n'y a pas d'engagement `SLA` de production dans le `MVP`. Un `SLA` pourra être défini en `V1+`.

— **Qualité :** le code doit rester clair, testé et documenté. La `CI/CD` se renforce au fil des versions.

4.11 API `REST` : principes de conception

— Le préfixe de version est `/api/v1`.

— Les échanges utilisent `JSON`.

— Les erreurs restent uniformes.

— Le schéma `OpenAPI` ou `Swagger` sert de référence pour le front et pour les tests.

— Les familles d'URL couvrent au minimum :
  — l'authentification ;
  — le catalogue ;
  — le panier ;
  — les commandes ;
  — les stocks, lots et alertes.

4.12 Conformité et protection des données

— Les mots de passe utilisent un hash robuste.

— Le système protège les sessions avec `JWT` et refresh token.

— Les entrées sont validées pour limiter les injections.

— Les règles `CORS` et les en-têtes de sécurité sont en place.

— Les actions sensibles sont journalisées.

— Les rôles suivent le principe du moindre privilège.

— Les données personnelles sont traitées pour le service, avec conservation et droits selon la politique SPLINEDGE et les exigences comme le `RGPD`.

4.13 Tests et validation

Stratégie de tests :

— tests unitaires pour les règles métier ;

— tests d'intégration pour l'API et la base ;

— tests `E2E` pour les parcours clés ;

— tests de non-régression pendant les évolutions.

Scénarios critiques :

— import de lot avec devise ;

— vente `FIFO` multi-lots ;

— rupture et alerte ;

— commande et statuts.

Données de test :

— jeux réalistes ;

— cas limites comme peu de stock, taux extrêmes et annulation.

Plan de validation par phase :

— `MVP` : validation des critères d'acceptation du CDC, avec auth, devise, `FIFO`, alertes, scan, commande et paiement ;

— `V1+` : renforcement de la sécurité, du paiement réel, de l'observabilité et de la recette métier élargie.

---

5. Organisation du projet

5.1 Ordre de développement recommandé

1. base projet et vitrine ;
2. connexion et rôles ;
3. catalogue et panier ;
4. commande et statuts ;
5. lots, devise et `FIFO` ;
6. multi-villes et alertes ;
7. scan ;
8. paiement selon la version ;
9. renforcement qualité : tests, sécurité et performance.

5.2 Planning indicatif

#	Étape
1	Cadrage détaillé
2	Réalisation `MVP`
3	Validation et ajustements
4	Préparation `V1`

5.3 Livrables par phase

— code source dans un dépôt versionné ;

— documentation technique et fonctionnelle ;

— jeux de tests et compte-rendu de validation ;

— guide d'exploitation initial.

5.4 Gouvernance projet

— Un point régulier, par exemple hebdomadaire, valide les jalons.

— Les changements importants demandent un arbitrage explicite.

— La version du CDC est incrémentée quand le périmètre ou un jalon majeur change. La proposition vient du développeur et la validation vient du superviseur.

5.5 Risques et plans de mitigation

Zone	Risque	Atténuation
Lots / `FIFO`	Erreurs de stock, erreurs de coût, bugs tardifs	Tests dédiés, recette métier et règles documentées
Périmètre	Trop de demandes parallèles	priorités MoSCoW figées par jalon et arbitrage du superviseur
Paiement	Retard d'intégration et doubles traitements	`MVP` simulé possible, sinon webhooks, idempotence et tests
Données	Démos peu fiables	Seeds reproductibles et contrôles de cohérence
Montée en charge	`MVP` trop limité	Modules découpés, API versionnée et mesure avant optimisation

Dépendances externes :

— disponibilité des décideurs pour la recette ;

— choix tranchés, par exemple Stripe oui ou non ;

— accès aux environnements et aux clés API.

5.6 Plan de démarrage projet

Environnements et accès

— L'initialisation reste centrée sur le `MVP`.

— Les exigences d'exploitation industrielle, comme `SLA` et `24/7`, seront détaillées plus tard.

Données initiales

— Les jeux de démo doivent être reproductibles avec des seeds alignés sur les scénarios de test du CDC.

Checklist de lancement

— dépôt et branches ;

— variables d'environnement ;

— base `PostgreSQL` ;

— démarrage des services avec `Docker Compose` ;

— migrations `Prisma` ;

— lancement front et back selon le `README` ;

— comptes de test pour les rôles `admin`, `customer` et `seller` ;

— décision `MVP` sur le paiement simulé ou Stripe.

---

6. Glossaire

Terme	Définition
AuthN	Vérifier qui est l'utilisateur.
AuthZ	Vérifier ce que l'utilisateur a le droit de faire.
FIFO	Premier entré, premier sorti, pour les lots de stock.
Moyenne pondérée (prix)	Prix de référence calculé à partir des prix par lot, pondéré par les quantités restantes.
Fournisseur	Tiers qui vend la marchandise à SPLINEDGE, avec conditions de paiement `crédit` ou `anticipé`.
Idempotence	Refaire la même opération ne crée pas un effet en double.
JWT	Jeton signé pour représenter une session ou des droits.
MVP	Première version limitée mais utilisable pour valider l'essentiel.
OpenAPI	Description formelle de l'API, souvent via `Swagger`.
RGPD	Règlement européen sur la protection des données personnelles.
SLA	Engagement sur la qualité de service, comme la disponibilité ou les délais.
Webhook	Message `HTTP` automatique envoyé par un service tiers, par exemple un service de paiement.